

July 2026 - 6

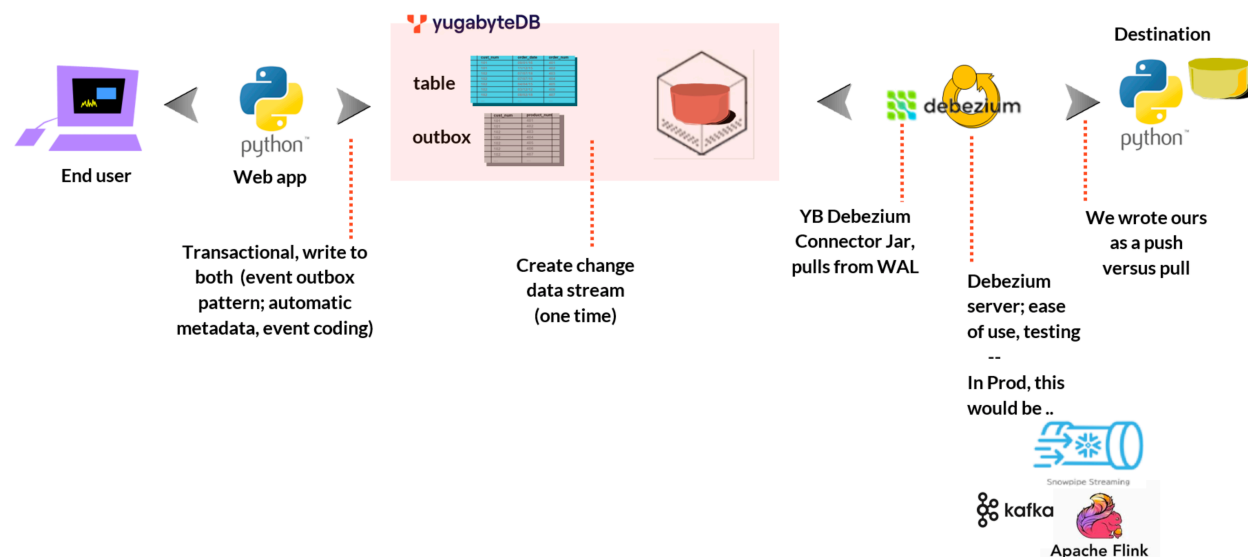


Figure 1: Figure 7-0: Change data capture in yugabyteDB

Welcome to this edition of yugabyteDB Developer's Notebook (YDN). This month we answer the following question(s);

My company wishes to understand the options for change data capture (CDC) when using yugabyteDB. We wish to avoid the cost of having to poll the database running given SQL SELECTS, to determine when conditions have changed that affect other portions of our business environment. Can you help ?

Excellent question ! There are two distinct CDC subsystems that come with yugabyteDB; yugabyteDB gRPC change data capture, and yugabyteDB (PostgreSQL) logical replication. Each has its application and use. In this article we will detail both systems and their configuration, testing, and related.

Software versions The primary software components used in this edition of YDN include yugabyteDB Anywhere (YBA) and yugabyteDB (YB) version 2025.2.2.2-b11. All of the steps below are run on one very large sized Mac Book Pro, or if you prefer, run these steps on yugabyteDB Aeon, yugabyteDB's managed service, or Amazon Web Services (AWS), Google Cloud Platform (GCP), or another hyper-scaler.

For isolation and (simplicity), we develop and test all systems inside virtual machines or emulated machines, using the desktop hypervisors VMware Fusion version 13.6.4, and/or UTM version 4.6.5. Generally we run a single node of YBA, and 8 nodes of YB (5 nodes in one region, and 3 nodes in a second region) using Alma Linux versions 8 and 9, and Ubuntu Desktop version 24.04. We also run a client node using Ubuntu Desktop version 24.04.

7.1 Terms and core concepts

7.1.1 Introduction: What Problem Does CDC Solve ?

Every application that stores data eventually faces the same challenge: other systems need to know when that data changes. A recommendation engine needs to react when a user updates preferences. A search index needs to reflect a new product record. An analytics warehouse needs the day's transactions. An audit log needs every modification to sensitive records. The list of downstream consumers that need to know about upstream changes grows with the application.

The naive solution is polling. You run a query that looks something like this:

Example 7-1: Polling to detect changes.

```
SELECT * FROM orders WHERE updated_at > :last_check_time;
```

You store the timestamp of the last run, schedule a job every few seconds, and compare. It feels simple. It works for a weekend project. But it fails in production in ways that compound over time.

The Polling Anti-Pattern The first failure mode is missed events. The polling approach depends entirely on the `updated_at` column being present, populated, and maintained correctly on every row that changes. A bulk `DELETE` has no `updated_at`. An `UPDATE` that resets a field back to its original value may look like no change at all if only one column carries the timestamp. Any row inserted and then deleted between two polling intervals vanishes from history entirely.

The second failure mode is database load. Polling is a full read operation against a table. At low frequency, the overhead is tolerable. At high frequency, you are running production reads against production tables on a recurring schedule, competing with application queries for buffer cache, I/O bandwidth, and CPU. When the table grows to hundreds of millions of rows, even a well-indexed range scan becomes a measurable cost.

The third failure mode is latency. A polling interval of ten seconds means that, in the worst case, a downstream consumer is ten seconds behind reality. For many use cases, ten seconds is acceptable. For fraud detection, order fulfillment, or real-time dashboards, it is not. Reducing the polling interval to one second multiplies the database load by a factor of ten and still leaves a one-second gap.

The fourth failure mode is ordering. Polling returns rows sorted by `updated_at`, which is the modification time, not the time the row was inserted or the order in which related rows were written. A parent row and its child rows may arrive out of order because they were updated at slightly different timestamps. Polling gives you a snapshot, not a change stream.

CDC as the Solution Change Data Capture solves these problems by tapping into the database’s own internal record of what happened, in the order it happened. Instead of asking “what has changed since I last looked,” CDC says “tell me every change as it occurs.” The source is not the table itself but the internal log the database uses to guarantee its own consistency.

This approach is event-driven rather than state-driven. Polling returns current state (rows as they exist now). CDC returns events (the sequence of changes that produced that state). This distinction matters enormously. If a record is inserted and then updated between two polling intervals, polling sees only the final state. CDC sees two events: an `INSERT` followed by an `UPDATE`.

The rest of this section builds the conceptual foundation needed to work with CDC in `yugabyteDB`: what the internal log is, how `yugabyteDB` is structured, what CDC paths are available, and the tools and patterns that tie everything together.

7.1.1.1 The Write-Ahead Log (WAL) Every major relational database uses a write-ahead log, commonly abbreviated WAL. Understanding the WAL is the single most important conceptual prerequisite for understanding CDC, because the WAL is the source that CDC reads from.

The term “write-ahead” describes the core rule: before any change is written to the actual data pages on disk, it must first be written to the log. This ordering guarantee is what makes the database durable. If the server crashes immediately after a commit but before the modified data pages have been flushed to disk, the database can reconstruct those changes by replaying the log entries on restart.

The WAL is a sequential, append-only record of every modification to the database in the exact order those modifications were committed. It records inserts, updates, and deletes at the row level. It records the before and after state of each row. It records transaction boundaries: when transactions began and when they committed or rolled back.

Because the WAL exists for durability and recovery, it is already being maintained at zero additional cost to the database. Every write that goes to the database goes through the WAL first. The WAL is not a feature you add for CDC. It is always there.

Durable Writes vs. In-Memory State It is worth being precise about what “written to the WAL” means in practice. When a client sends a COMMIT statement, the database does not immediately flush every modified data page from memory to disk. Flushing data pages is expensive. Instead, the database flushes the WAL record for that transaction to disk. The WAL record is small and sequential, making it fast to write.

[!NOTE] A transaction is considered durable the moment its WAL record is on disk. The actual data pages can be written later, during a background checkpoint. This is why databases can acknowledge a commit immediately and still guarantee durability: the WAL is the definitive record. In-memory state (buffer cache) may be newer than disk, but the WAL always reflects committed reality.

This distinction matters for CDC because CDC reads from the WAL, not from the in-memory buffer cache and not from the data pages. A CDC consumer reading the WAL is reading from a durable, ordered, complete record of all committed changes. newer than disk, but the WAL always reflects committed reality.

The WAL as the Ideal CDC Source The WAL has properties that make it uniquely well-suited as a CDC source. First, it is complete. Every committed row-level change is in the WAL, regardless of whether the affected row has an `updated_at` column or how the change was made. Bulk deletes, DDL operations, upserts – they are all in the WAL.

Second, it is ordered. WAL records appear in commit order. Within a transaction, row changes appear in the order the statements executed. Across transactions, the WAL reflects the linearized commit sequence.

Third, it is independent of the application. CDC reads the WAL without touching the application tables directly. There is no additional load on the tables being monitored. A CDC process reading the WAL does not interfere with queries, updates, or other reads against those same tables.

Fourth, the WAL includes all row versions. An UPDATE event in the WAL contains both the before-image (the row as it was) and the after-image (the row as it became), subject to how `REPLICA IDENTITY` is configured on the table. This before/after data is what enables downstream systems to detect exactly what changed, not just that something changed.

7.1.1.2 yugabyteDB Architecture Basics Before examining how CDC works in yugabyteDB specifically, it is helpful to understand the basic structure of a yugabyteDB cluster. yugabyteDB is a distributed SQL database that provides PostgreSQL compatibility at the query layer while running a distributed storage engine underneath.

PostgreSQL Compatibility yugabyteDB supports the PostgreSQL wire protocol, the PostgreSQL SQL dialect, and most PostgreSQL extensions and features. Applications written for PostgreSQL generally connect to yugabyteDB without modification. This compatibility extends to the system catalog tables that tools use to interact with replication features: `pg_replication_slots`, `pg_create_logical_replication_slot()`, and related functions are all present and functional.

This compatibility is significant for CDC because it means that tools designed for PostgreSQL logical replication work with yugabyteDB with little or no modification. The PostgreSQL ecosystem of connectors, pipelines, and frameworks applies directly.

DocDB: The Storage Engine Below the PostgreSQL compatibility layer, yugabyteDB runs DocDB, its own distributed storage engine. DocDB is based on RocksDB, a key-value store that uses a Log-Structured Merge-tree (LSM-tree) architecture. In an LSM-tree, writes always go to an in-memory buffer

(the MemTable) and are periodically flushed to sorted files on disk (SSTables). Reads merge the in-memory and on-disk data to produce a consistent result.

The LSM-tree architecture is well-suited to write-heavy workloads and distributed systems. It enables high write throughput without the random I/O patterns that B-tree indexes generate. It also enables efficient compaction, which is the background process that merges SSTables to reclaim space and improve read performance.

Tablets: The Unit of Distribution yugabyteDB distributes data across a cluster by dividing each table into tablets. A tablet is a range partition of a table's data, stored and served by a specific node. When a table is created, yugabyteDB automatically splits it into multiple tablets based on the cluster size and configured split parameters.

Tablets are the granular unit of replication, load balancing, and CDC event production. In a CDC context, each tablet independently produces a stream of change events for the rows it holds. A table with 30 tablets produces 30 independent change streams that can be consumed in parallel.

[!NOTE] The number of tablets matters for CDC throughput. In the gRPC CDC path, consumers can read from all tablets in parallel. In a 100-node cluster with approximately 20,000 tablets, this parallel consumption is what enables the 250,000 records/sec throughput target. The PostgreSQL Logical Replication path, by contrast, funnels changes through a single coordinator node regardless of the tablet count, which is why its current throughput ceiling is lower.

TServers: Tablet Servers Tablet servers (TServers) are the nodes in a yugabyteDB cluster that store and serve data. Each TServer hosts some number of tablets. Reads and writes from client applications go to the TServer that holds the relevant tablet. For distributed transactions, TServers coordinate with each other to maintain consistency.

Each TServer runs an instance of DocDB. For CDC, TServers are the origin of change events: when a row is modified in a tablet, the change is recorded in that tablet's WAL-equivalent structure, and the CDC layer reads from that structure.

Masters: The Coordination Layer The master servers handle metadata, DDL operations, and routing. Clients ask the masters which TServer holds a given tablet. DDL operations like CREATE TABLE and ALTER TABLE go through the masters. The masters maintain the system catalog and ensure that the cluster's tablets are properly replicated and distributed.

There is typically one active master leader and two or more standby masters (in a 3-node cluster with RF=3, all three nodes run both a TServer and a master). For CDC, the master addresses are needed when setting up the gRPC CDC path, because the CDC stream is registered at the master level.

Replication Factor yugabyteDB replicates each tablet across multiple nodes to provide fault tolerance. The replication factor (RF) specifies how many copies of each tablet exist. With RF=3, every tablet has three copies on three different nodes. Reads can be served from any replica. Writes go through a Raft consensus protocol that requires a majority of replicas to acknowledge the write before committing.

The standard recommendation for production clusters is RF=3. This tolerates the loss of one node (one out of three replicas) without any data loss or unavailability. For CDC, the replication factor does not change the number of change events produced. Each change event is generated once, from the Raft leader of the tablet, regardless of the replication factor.

Figure: 7-1 displays an architecture diagram relative to yugabyteDB RAFT protocol. A code review follows.

Relative to Figure: 7-1, the following is offered:

- The image above displays a three node yugabyteDB universe. Because we are discussing replication, and tables/tablets, we will say that the image above is displaying yugabyteDB “tservers” (tablet servers) only, and that no yugabyteDB “master” servers are displayed. With yugabyteDB, a table is a logical

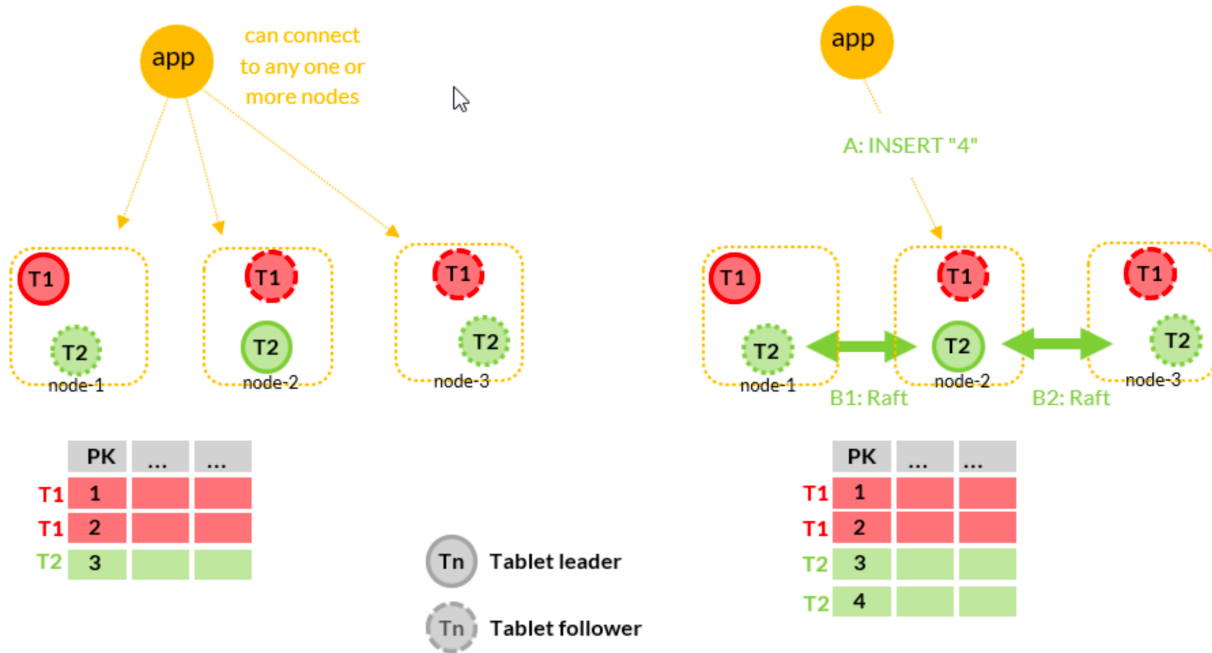


Figure 2: Figure 7-1: yugabyteDB RAFT protocol

entity, comprised of one or more physical entities that are tablets. Tablets are the unit of storage placed on disk.

- A given table “T”, was initially created and pre-split into two tablets (physical partitions). From the image, we can not determine if the table is partitioned by hash, or by range. We imagine that the table was created using pre-splitting, since the row count is so low, this table would not have experienced splitting due to growth.
- On the left side of the image, a client program maintains network connections to all tservers. This is configurable. The client could have connected to one tserver only. When the client initially connects to the cluster, and then periodically throughout its life cycle, the driver fetches cluster metadata, including; all nodes in the cluster, partition key ranges per table, and tablet/tserver leader mappings.
- On the right side of the image, the client calls to SQL INSERT a new row with a primary key of “4”. The driver determines that the tablet leader for this hash or range value is tablet T2. The client sends the write request to node-2, tablet leader T2 only. The driver handles this transparently. For writes, the client has no knowledge or interaction with the tablet followers.
- The client only talks to the leader. The driver handles this transparently. It knows which tserver holds the tablet leader for that row’s partition key, and routes the write directly there. The client has no knowledge of or interaction with the followers. The leader does the fan-out. After writing locally, the leader is responsible for replicating to the two followers via the Raft protocol. The client is not involved in this at all. Quorum acknowledgment. With RF=3, yugabyteDB needs two of three nodes to confirm before the write is acknowledged to the client. So the client waits for the leader + 1 follower to confirm, not all three. This means:

If one follower is slow, you don’t wait for it. If one tserver dies entirely, writes still succeed.

The third follower catches up asynchronously if it was slow or briefly unavailable.

7.1.2 The Two CDC Paths in yugabyteDB

yugabyteDB provides two distinct mechanisms for capturing and consuming change data. They differ in protocol, tooling, performance characteristics, and ecosystem compatibility. Understanding both allows you to choose the right path for a given use case.

The first path is gRPC CDC, yugabyteDB’s native high-performance CDC protocol. It reads directly from tablets in parallel using a proprietary gRPC-based protocol and is the choice for maximum throughput requirements.

The second path is PostgreSQL Logical Replication, introduced in yugabyteDB 2024.1. It uses the standard PostgreSQL replication protocol and is compatible with any tool that supports PostgreSQL logical replication, including Debezium, Striim, Fivetran, and direct Python connections via psycopg2.

7.1.2.1 gRPC CDC (The Native Path) gRPC CDC is yugabyteDB’s original CDC mechanism, available since early versions of the product. It was designed for high-throughput streaming at cluster scale and exposes the parallel, tablet-level architecture of yugabyteDB directly to the CDC consumer.

The central concept is the CDC stream. A CDC stream is a cluster-level object, created against an entire database, that enables change capture for all tables (or a specified subset of tables) in that database. The stream is created using the yb-admin command-line tool, which ships with every yugabyteDB installation.

Creating a gRPC CDC Stream To create a CDC stream, you run the `create_change_data_stream` subcommand of yb-admin, specifying the master addresses and the database name:

Example 7-2: Creating a gRPC change data capture stream

```
yb-admin \  
--master_addresses master1:7100,master2:7100,master3:7100 \  
create_change_data_stream ysql.<database_name>
```

The prefix “ysql.” indicates that this is a YSQL (PostgreSQL-compatible) database as opposed to a YCQL (Cassandra-compatible) database. yugabyteDB supports both APIs; for our purposes we use YSQL exclusively.

The command returns a Stream ID, which is a UUID-format identifier:

Example 7-3: Change data capture stream id

CDC Stream ID: d540f5e4802b4d9589aeedea62d03079

This Stream ID is used in the Debezium connector configuration to tell Debezium which CDC stream to consume. It is also used to inspect and manage the stream using yb-admin administrative commands.

Parallel Tablet Consumption Once a CDC stream exists, it can be consumed in parallel across all tablets. The Debezium connector for yugabyteDB gRPC reads from each tablet leader independently and in parallel. The consumer is responsible for merging or routing the per-tablet streams.

In a large cluster, this parallelism produces substantial throughput. Shopify, which contributed significantly to yugabyteDB’s gRPC CDC development, targets 250,000 records per second across a 100-node cluster with approximately 20,000 tablets. This throughput is achievable precisely because the consumption is distributed: no single node or coordinator is a bottleneck.

The gRPC Protocol The underlying protocol is proprietary gRPC. While gRPC is a general-purpose RPC framework, the specific message formats and APIs used by yugabyteDB’s CDC path are yugabyteDB-specific. This means that only connectors explicitly written for yugabyteDB’s gRPC CDC API can consume this path.

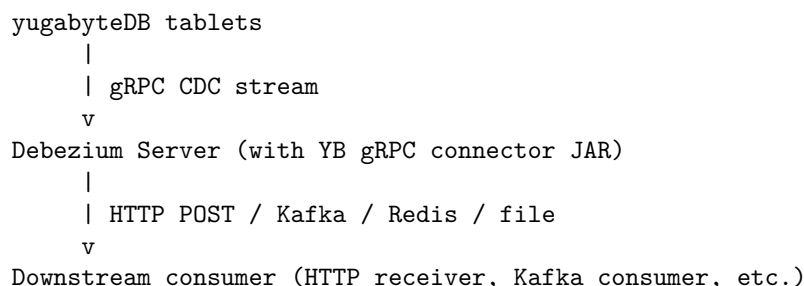
In practice, this means Debezium with the yugabyteDB gRPC connector JAR. The connector is maintained in the yugabyte/debezium-connector-yugabytedb repository on GitHub. It is pinned to Debezium 1.9.5,

which is an older stable version of Debezium. (The current Debezium release line, based on 2.5.x and later, is used for the PostgreSQL Logical Replication path, covered in section 1.2.2.)

Debezium is covered in detail in section 1.4. For the gRPC path, the key point is that Debezium Server acts as the CDC consumer: it connects to the yugabyteDB cluster, reads the gRPC CDC stream, transforms change events, and delivers them to a configured sink (HTTP, Kafka, Redis, file, and others).

Example 7-4: gRPC change data capture architecture overview

The end-to-end architecture for gRPC CDC is:



The Debezium Server process is a standalone Java application. It requires Java 21. It reads change events from the gRPC CDC stream, wraps them in Debezium’s standard change event envelope format, and delivers them to the configured sink.

Figure: 7-2 displays a data flow diagram when using gRPC. A code review follows.

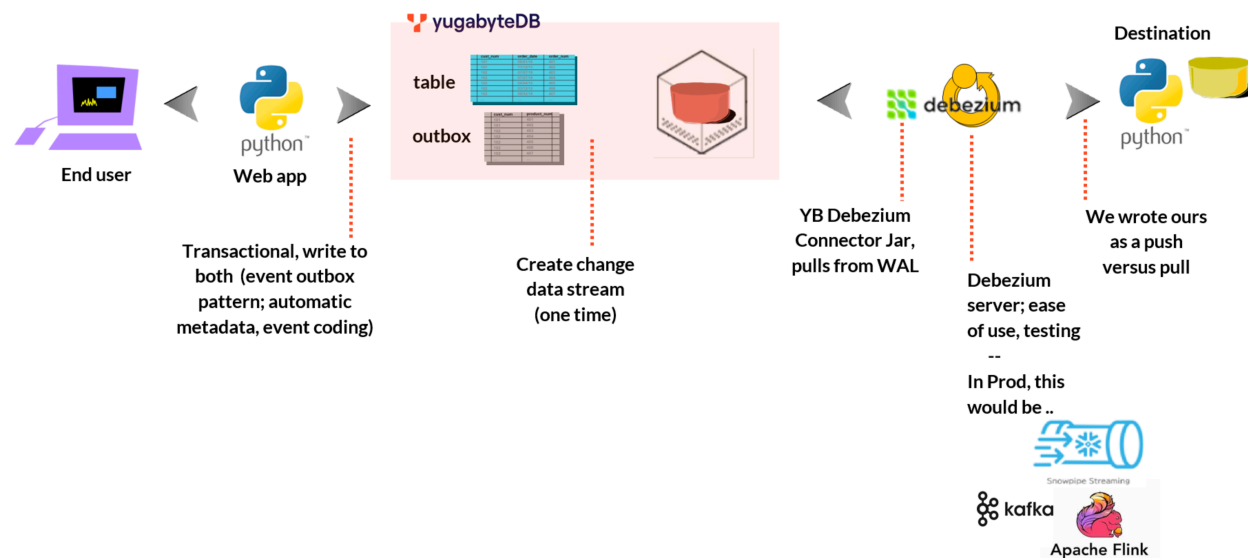


Figure 3: Figure 7-2: gRPC change data capture flow diagram

7.1.2.2 PostgreSQL Logical Replication (The Compatible Path) PostgreSQL Logical Replication support was introduced in yugabyteDB 2024.1. This path implements the standard PostgreSQL logical replication protocol, making yugabyteDB compatible with the entire ecosystem of tools that support PostgreSQL replication, without requiring any yugabyteDB-specific connectors or libraries.

The standard PostgreSQL logical replication protocol is well-documented, widely implemented, and supported by a large number of commercial and open-source data integration tools. Adding this capability to yugabyteDB means that organizations with existing PostgreSQL tooling can use those same tools with yugabyteDB.

Prerequisites: wal_level and Replication Privilege For PostgreSQL Logical Replication to work, two conditions must be met. First, the wal_level database configuration parameter must be set to 'logical'. This tells the database to include additional information in the WAL that logical replication consumers need – specifically, enough information to reconstruct the row-level changes (the “logical” view) rather than the raw physical block changes.

You can verify the current wal_level setting with:

Example 7-5: SQL command to report the currently effective wal_level setting.

```
SHOW wal_level;
```

If the result is not 'logical', the flag must be set and the cluster restarted. In yugabyteDB, this is a gflag (-ysql_enable_replication_slot=true also applies). Consult the yugabyteDB documentation for the current procedure for your version.

Second, the database user used for replication must have the REPLICATION privilege. The built-in yugabyte superuser typically has this by default. You can verify with:

Example 7-6: SQL command to verify authorization level.

```
SELECT rolname, rolreplication FROM pg_roles WHERE rolname = 'yugabyte';
```

Replication Slots Unlike the gRPC CDC path, which uses a cluster-level stream object, PostgreSQL Logical Replication uses replication slots. A replication slot is a server-side bookmark into the WAL that tracks how far a specific consumer has read. Slots are covered in detail in section 1.3; for now, the important point is that creating and managing slots requires only SQL, not CLI tools.

To create a replication slot using the wal2json output plugin:

Example 7-7: SQL to create logical replication slot.

```
SELECT pg_create_logical_replication_slot('my_cdc_slot', 'wal2json');
```

This is a standard SQL statement that any PostgreSQL client can execute. No yb-admin, no cluster-level configuration, no Java. Once the slot exists, a consumer can connect using the standard PostgreSQL replication protocol and start receiving change events.

No yb-admin Required This is one of the most practically significant differences between the two paths. The PostgreSQL Logical Replication path requires no yugabyteDB-specific administrative tools. A developer with a psql client, a PostgreSQL connection string, and Python with psycopg2 installed can set up and consume a CDC stream using only SQL statements.

This substantially reduces the operational overhead of CDC for teams that are already comfortable with PostgreSQL tooling. It also means that the CDC setup can be embedded in application initialization scripts, database migration tools, or infrastructure-as-code systems without requiring special tooling.

[!NOTE] Output Plugins

PostgreSQL Logical Replication uses output plugins to format the change events that are sent to consumers. yugabyteDB ships with two output plugins:

- wal2json:** Formats change events as JSON. Human-readable and easy to parse with standard JSON libraries. This is the plugin used in Demo B. Covered in detail in section 1.5.
- pgoutput:** PostgreSQL's native binary replication protocol. Used by tools that speak the native PostgreSQL replication protocol at a binary level, such as certain versions of Debezium's PostgreSQL connector.

For most new projects, wal2json is the recommended starting point. The JSON output is transparent, debuggable, and requires no special decoding libraries.

Current Performance and Roadmap The current throughput of PostgreSQL Logical Replication in yugabyteDB is approximately 5,000 records per second per replication slot. This limitation exists because, regardless of how many tablets a table is distributed across, the logical replication output is coordinated through a single node. The distributed parallelism of the gRPC CDC path is not present here.

The bottleneck is architectural: the PostgreSQL logical replication protocol was designed for a single-node database, and adapting it to a distributed system while maintaining protocol compatibility requires coordination overhead. The yugabyteDB engineering team has this on their roadmap and is targeting approximately 25,000 records per second in the yugabyteDB 2026.2 release.

[!NOTE] Creating multiple replication slots does NOT partition the throughput. If you create three slots watching the same table, you get three duplicate streams, each carrying the same 5,000 records/sec. You do not get one stream partitioned across three consumers at 15,000 records/sec total. Horizontal scaling of PG Logical Replication throughput is a future capability, not a current one. For high-throughput requirements today, use the gRPC CDC path.

Compatible Tools

Because the PostgreSQL Logical Replication protocol is standard, any tool that can consume PostgreSQL logical replication works with yugabyteDB. This includes:

- Debezium with the PostgreSQL connector (using yugabyte/debezium repo, which adds yugabyteDB-specific support to Debezium 2.5.2+)
- Striim
- Fivetran
- Airbyte
- AWS Database Migration Service (DMS)
- psycopg2 directly (Python, no Java, no Kafka, no Debezium required)
- Any other tool with PostgreSQL logical replication support

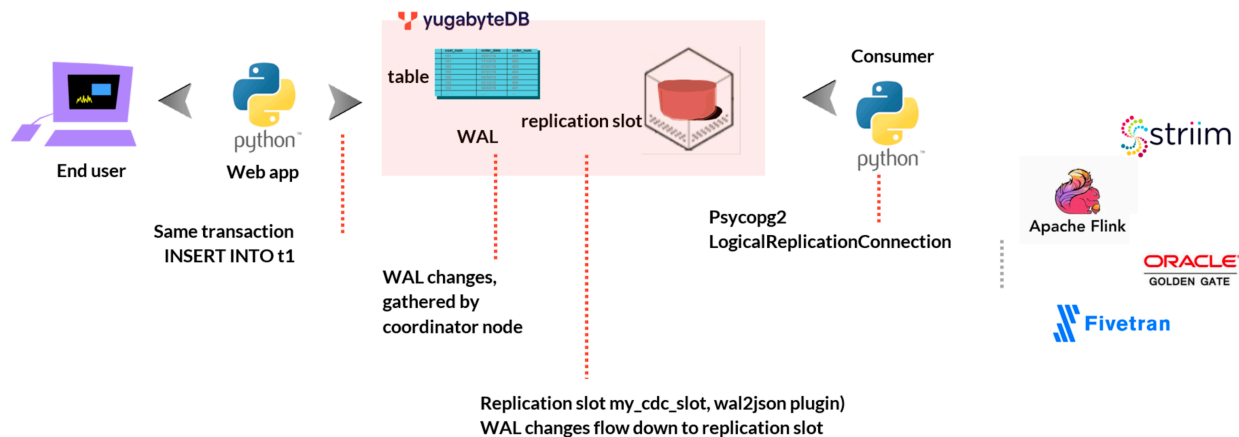


Figure 4: Figure 7-3: Logical change data capture flow diagram

7.1.2.3 Comparing the two paths Example 7-8: Comparing the two paths

The table below summarizes the key differences between the gRPC CDC path and the PostgreSQL Logical Replication path.

+-----+-----+-----+-----+

Aspect	gRPC CDC	PG Logical Replication
Protocol	Proprietary YB gRPC	Standard PostgreSQL
Setup	yb-admin CLI	SQL (pg_create_logical...)
Connector	YB gRPC Debezium JAR	Any PG-compatible tool
Java required?	Yes (Debezium Server)	No (can use psycopg2)
Throughput	~250K records/sec	~5K records/sec (today)
Ordering	Per-tablet (parallel)	Single ordered stream
Ecosystem	yugabyteDB only	Full PostgreSQL ecosystem
Debezium version	1.9.5 (pinned)	2.5.2+

Example 7-9: When to Choose gRPC CDC

Choose the gRPC CDC path when:

- Throughput is the primary concern. If you need to stream hundreds of thousands of records per second, gRPC CDC is the only current option that delivers that performance in yugabyteDB.
- You are operating a large cluster (tens to hundreds of nodes) and want to take advantage of the parallel, per-tablet consumption model.
- Your organization is standardized on Debezium and already has Java infrastructure for running Debezium Server or Kafka Connect.
- You are integrating with Kafka and want Debezium's rich transformation and routing capabilities.

Example 7-10: When to Choose PostgreSQL Logical Replication

Choose the PostgreSQL Logical Replication path when:

- Throughput requirements are below 5,000 records/sec per stream, which covers the majority of real-world OLTP applications.
- You want to minimize infrastructure complexity. No Java, no Debezium, no Kafka. A Python script with psycopg2 is sufficient.
- You are migrating from PostgreSQL and want to reuse existing CDC tooling and expertise.
- You want the broadest possible choice of integration tools, including commercial ETL platforms.
- You are building a demonstration or prototype and want the quickest path to seeing change events.

[!NOTE] For many production applications, the 5,000 records/sec ceiling of PostgreSQL Logical Replication is not a constraint. A busy e-commerce application might write 100-500 rows per second at peak. Only applications at significant scale (high-volume payment processing, IoT data ingestion, large-scale event sourcing) are likely to hit the current throughput ceiling. Measure your actual write rate before assuming you need gRPC CDC's throughput.

The performance roadmap is also relevant to this decision. yugabyteDB 2026.2 is targeting 25,000 records/sec for PostgreSQL Logical Replication. If your throughput requirements are

under 25,000 records/sec and you are building for deployment in the second half of 2026 or later, PostgreSQL Logical Replication may be entirely sufficient for your needs.

7.1.3 Replication Slots

A replication slot is a server-side object that tracks the progress of a logical replication consumer. Understanding slots is essential for operating CDC in production, because slots directly affect disk usage, availability, and failover behavior.

The fundamental concept is that a replication slot is a bookmark into the WAL. It records the WAL position up to which a specific consumer has acknowledged consuming events. The database uses this position to determine which WAL records can be discarded (because no slot needs them anymore) and which must be retained (because at least one slot still needs them).

This WAL retention behavior is both a feature and a risk. It is a feature because it means that if a consumer disconnects and reconnects, it picks up exactly where it left off, with no missed events. It is a risk because if a consumer disconnects and does not reconnect, the WAL grows indefinitely as the database retains records for the inactive slot. On a busy database with high write throughput, an inactive slot can consume significant disk space in a short time.

One Slot Per Consumer A replication slot corresponds to one logical consumer, not to one table and not to one tablet. A consumer that wants to receive changes from multiple tables uses a single slot that covers all of those tables. Multiple consumers each need their own slot.

It is important to understand that creating a second slot watching the same table does not partition the events between the two slots. Each slot receives a complete, independent copy of every change event for the tables it covers. Two slots watching the same table means two identical streams, not one stream split between two consumers. Use multiple slots when you have multiple independent consumers that each need to see every event, not when you want to share the load of processing a single stream.

Creating a Replication Slot Creating a slot requires a single SQL statement. You specify a slot name (any valid identifier) and an output plugin (wal2json or pgoutput):

Example 7-11: Creating a replication slot

```
SELECT pg_create_logical_replication_slot('my_cdc_slot', 'wal2json');
```

The result is a row with two columns: the slot name and the WAL position at which the slot was created:

Example 7-12: Result of creating a replication slot

slot_name	lsn
my_cdc_slot	0/3000000

From this point forward, the slot begins retaining WAL records. Even if no consumer connects, the slot accumulates unapplied changes.

Listing Replication Slots To see all replication slots and their current state:

Example 7-13: SQL SELECT to verify all replication slots and their current state

```
SELECT * FROM pg_replication_slots;
```

The important columns to check are: acknowledged events

Example 7-14: Output from SQL SELECT above.

slot_name:	The slot's identifier
active:	Whether a consumer is currently connected

`restart_lsn`: The oldest WAL position the database must retain for this slot (how far behind the slot is)
`confirmed_flush_lsn`: The WAL position up to which the consumer has acknowledged events

Dropping a Replication Slot To remove a slot that is no longer needed:

Example 7-15: Dropping a replication slot

```
SELECT pg_drop_replication_slot('my_cdc_slot');
```

[!NOTE] You cannot drop an active replication slot. If a consumer is currently connected and consuming from the slot, the drop command will fail. In yugabyteDB, a slot remains “active” for up to five minutes after the last consumer disconnects, controlled by the GFlag `ysql_cdc_active_replication_slot_window_ms` (default: 300000 ms). This means you may need to wait up to five minutes after stopping a consumer before you can drop its slot. In reset scripts for development environments, this is handled by adding a `pg_sleep(5)` or retry loop before the drop.

In practice, dropping a slot is most commonly needed during development (to reset the CDC stream position), during schema migrations (to recreate a slot with new options), or when de-commissioning a consumer.

Disk Space and WAL Retention On a production cluster with significant write throughput, an inactive slot is a monitoring concern. If a CDC consumer goes down unexpectedly and the alert is not caught, the WAL will grow for as long as the consumer is absent. On a write-intensive database, this can exhaust disk space within hours or days.

Best practices for managing this risk include:

- Set up monitoring alerts on `pg_replication_slots` to detect inactive slots with a large lag.
- Configure a maximum WAL size limit (this varies by yugabyteDB version).
- Use infrastructure-as-code to manage slot life cycle. Create slots when consumers are deployed, drop them when consumers are removed.
- In development and testing environments, use reset scripts that always drop and recreate slots at the start of each test run.

7.1.4 Debezium

Debezium is an open-source Change Data Capture platform originally created by Red Hat and now a major project in the CDC ecosystem. It provides a unified framework for reading change events from various database systems, transforming those events into a standard format, and routing them to downstream destinations.

Debezium was designed from the beginning around the concept of reading from database WALs and change streams rather than polling. It supports a wide range of source databases: PostgreSQL, MySQL, MongoDB, Oracle, SQL Server, Db2, and others. It connects to each database’s native replication mechanism and produces change events in a consistent format regardless of the source.

For yugabyteDB, there are two Debezium connector repositories, one for each CDC path:

- For gRPC CDC:
Repository: `yugabyte/debezium-connector-yugabytedb`
Based on: Debezium 1.9.5 (pinned)

Protocol: gRPC CDC (proprietary)

```
-- For PostgreSQL Logical Replication:
Repository: yugabyte/debezium
Based on: Debezium 2.5.2+
Protocol: PostgreSQL logical replication (standard)
```

These are maintained separately because they use different Debezium core versions and different connection protocols. When you see documentation or examples for Debezium with yugabyteDB, confirm which path and which repository is being referenced, as the configuration and setup differ significantly.

Two Deployment Modes

Debezium can be deployed in two modes:

Debezium Server: A standalone Java process that runs Debezium without Kafka. Change events are read from the source database and written directly to a configured sink: HTTP, Redis, Google Pub/Sub, Amazon Kinesis, Apache Pulsar, or a local file. This is the mode used in Demo A.

Kafka Connect: Debezium runs as a set of Kafka Connect connectors within a Kafka Connect cluster. Change events are written to Kafka topics and consumed by downstream Kafka consumers. This mode requires Kafka infrastructure but provides durability, consumer group semantics, and Kafka's full stream processing ecosystem.

For teams that do not have or do not want Kafka infrastructure, Debezium Server is the lighter-weight option. The HTTP sink is particularly useful: it delivers each change event as an HTTP POST to a configured URL, making it easy to receive events in any web application without any Kafka client library.

The Debezium Change Event Envelope

Example 7-16: The Debezium change event envelope

Debezium wraps every change event in a standard envelope format. The envelope structure includes:

before:	The row's state before the change. Null for INSERT events.
after:	The row's state after the change. Null for DELETE events.
source:	Metadata about the source: database name, table name, timestamp, transaction ID, and connector-specific fields.
op:	The operation type. 'c' for create (INSERT), 'u' for update, 'd' for delete, 'r' for read (initial snapshot), 't' for truncate.
ts_ms:	The event timestamp in milliseconds since epoch.
transaction:	If <code>provide.transaction.metadata</code> is enabled, the transaction ID, total event count, and per-table event order.

This consistent envelope format means that code written to process Debezium events from one database can be adapted to process events from a different database with minimal changes.

Sink Options

Debezium Server supports the following sink types (as of version 1.9.5 and later versions):

```
-- HTTP: POST each event to a URL
-- Apache Kafka: publish to a Kafka topic
-- Google Cloud Pub/Sub: publish to a GCP Pub/Sub topic
-- Amazon Kinesis: publish to a Kinesis data stream
-- Apache Pulsar: publish to a Pulsar topic
-- Redis: publish to a Redis stream
-- File (for testing and debugging)
```

For Demo A in this document, the HTTP sink is used. Debezium Server posts each change event to a Flask web application running on the same machine. This setup is appropriate for demonstration purposes and for low-to-medium throughput production use cases where the receiver can keep up with the event rate.

[!NOTE] Java 21 is required for the version of Debezium Server used in this document. Java 25 (which may be on your system if you have a recent JDK installed) is not tested with Debezium 1.9.5 and may cause compatibility issues. Use Java 21 specifically. The installation steps in section 2.2.4 use Eclipse Temurin 21 LTS, which is a reliable, freely available OpenJDK distribution.

7.1.5 wal2json

wal2json is a PostgreSQL output plugin that converts WAL records into JSON format. It is bundled with yugabyteDB and is available for use as soon as the `wal_level` is set to 'logical'. No additional installation is required.

The purpose of wal2json is to provide a human-readable, machine-parseable representation of WAL change events. Raw WAL records are binary and database-internal. wal2json translates those binary records into JSON objects that application code can process with a standard JSON library.

What wal2json Produces wal2json operates at the row level, not the statement level. A single UPDATE statement that modifies 1,000 rows produces 1,000 separate wal2json events, one for each affected row. This granularity is essential for CDC consumers that need to process individual row changes, not bulk operations.

Example 7-17: Each wal2json event includes the following fields by default:

xid:	The transaction ID of the transaction that made this change
change:	An array of row-level change objects
kind:	The operation type: 'insert', 'update', or 'delete'
schema:	The schema name (typically 'public')
table:	The table name
columnnames:	Array of column names
columnvalues:	Array of column values (for INSERT and UPDATE after-state)
oldkeys:	For UPDATE and DELETE, the identifying key columns and values (present when REPLICA IDENTITY is configured appropriately)

Additional fields can be requested via options passed to `start_replication`. The most useful are `include-xids` (include the transaction ID) and `include-timestamp` (include the commit timestamp).

Example: An INSERT Event Example 7-18: Here is a representative wal2json output for a single INSERT into table `t1`:

```
{
  "xid": 1234,
  "change": [
    {
      "kind": "insert",
      "schema": "public",
      "table": "t1",
      "columnnames": ["col1", "col2", "col3", "col4"],
      "columnvalues": ["Alice", "Blue", "Large", "Active"]
    }
  ]
}
```

The `columnnames` and `columnvalues` arrays are parallel: `columnnames[0]` corresponds to `columnvalues[0]`, and so on. A consumer iterates both arrays together to reconstruct the row.

Example: A DELETE Event For DELETE events, wal2json needs to know which columns to include in the event. By default, PostgreSQL includes only the primary key columns in a DELETE event. This is sufficient to identify the deleted row by its key, but does not tell the consumer what the deleted row's other values were.

Example 7-19: To include all column values in DELETE events, set **REPLICA IDENTITY FULL** on the table:

```
ALTER TABLE t1 REPLICA IDENTITY FULL;
```

Example 7-20: With **REPLICA IDENTITY FULL**, a DELETE event includes all column values in the oldkeys section:

```
{
  "xid": 1238,
  "change": [
    {
      "kind": "delete",
      "schema": "public",
      "table": "t1",
      "oldkeys": {
        "keynames": ["col1", "col2", "col3", "col4"],
        "keyvalues": ["Alice", "Blue", "Large", "Active"]
      }
    }
  ]
}
```

[!NOTE] A single DELETE FROM t1 WHERE col3 = 'Large' that affects 50 rows produces 50 separate wal2json change events, each describing one deleted row. Your consumer must be prepared to handle a burst of individual DELETE events for what was a single SQL statement. This is the correct behavior for row-level CDC: the consumer sees individual row changes, not the statement that caused them.

wal2json Format Versions Example 7-21: wal2json supports two format versions:

Version 1 (default): One JSON message per transaction, containing an array of all row changes in the transaction in the "change" array. All rows from the same transaction arrive in one message.

Version 2: One JSON message per row change. Each message contains exactly one row change and includes the transaction information. This mode is useful when you want to process row changes as a stream of individual events rather than grouped by transaction.

The examples in this document use the default (version 1) format unless otherwise noted.

7.1.6 The Outbox Pattern

The Outbox Pattern is a software design pattern that addresses a fundamental reliability problem in event-driven systems: the dual-write problem. It is not CDC-specific, but CDC is the standard implementation mechanism for the Outbox Pattern in modern systems.

The Dual-Write Problem

Consider a service that does the following when processing an order:

1. INSERT a new row into the orders table
2. Publish an OrderCreated event to a message bus (Kafka, RabbitMQ, etc.)

These are two separate I/O operations to two separate systems. Either one can fail independently of the other. If step 1 succeeds but step 2 fails, the order is in the database but no one was notified. The event is permanently lost unless the application has explicit retry logic, and even with retry logic, the retry mechanism itself can fail.

The converse failure is equally bad: if step 2 succeeds but step 1 fails (or is rolled back), the event has been published for a row that does not exist. Downstream consumers will try to process an order that is not in the database.

This is the dual-write problem: atomically committing to two separate systems is only possible with distributed transactions (two-phase commit), which are complex, slow, and operationally fragile. Most teams correctly choose to avoid two-phase commit.

The Outbox Solution The Outbox Pattern eliminates the dual-write problem by making both writes go to the same system (the database) in a single transaction. Instead of publishing directly to the message bus, the application writes to a dedicated outbox table. The outbox table is inside the same database, subject to the same ACID transaction that writes the business data.

Example 7-22: The flow looks like this:

```
BEGIN TRANSACTION
-- Write the business data
INSERT INTO orders (id, customer, total) VALUES (...);

-- Write the event to the outbox (same transaction)
INSERT INTO outbox_events (aggregate_type, event_type, payload) VALUES (...);

COMMIT -- Both writes succeed or both fail
```

A CDC process watches the outbox table. When a new row appears in the outbox, the CDC process delivers it to the message bus (or directly to downstream consumers). The CDC process reads from the WAL, which means it sees the event only after the transaction has committed. If the transaction rolls back, no row appears in the outbox, and no event is published.

The key insight is that CDC provides the delivery mechanism, while the database transaction provides the atomicity guarantee. You get exactly-once event publication relative to the database operation.

Why Use the Outbox Pattern Even When You Have Direct CDC ?

Some developers ask: if CDC already gives you every change to every table, why add an outbox table at all? Why not just CDC the business table directly ?

Example 7-23: There are several good reasons to prefer the outbox pattern:

Schema isolation:	Direct CDC exposes your internal table schema to downstream consumers. If you rename a column, add a column, or change a data type, every consumer breaks. The outbox table is a designed API: you control exactly what fields appear in the payload, and you can evolve it independently of the underlying table schema.
Event design:	Not every row change corresponds to a business event. An UPDATE that increments a <code>retry_count</code> column is not the same business event as an UPDATE that changes an order's status from 'pending' to 'shipped'. The outbox lets you be explicit about which changes constitute

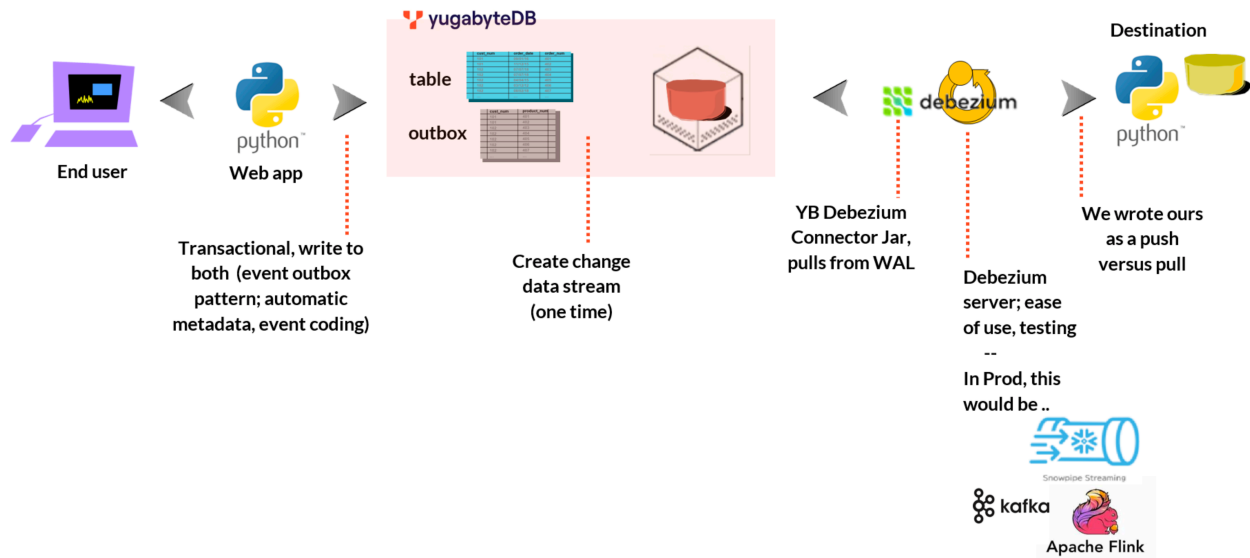


Figure 5: Figure 7-4: Outbox pattern

events and what those events mean.

Multiple aggregates: A single business operation may modify multiple tables. The outbox provides a single stream of business events regardless of how many tables were affected.

Event enrichment: The outbox payload can include computed values, denormalized data, or application-level metadata that does not appear in any single table.

Idempotency keys: The outbox row can carry explicit idempotency keys, making it easier for consumers to detect and ignore duplicate deliveries.

[!NOTE] CDC does not require the Outbox Pattern. The Outbox Pattern uses CDC as its delivery mechanism. These are independent concepts that work well together. You can use CDC without an outbox (watching business tables directly) and you can implement an outbox without CDC (using polling on the outbox table). The combination of CDC and outbox is simply the most reliable and scalable approach for event-driven systems.

The Outbox Table Schema Example 7-24: The outbox table schema can be customized for your needs. A standard starting point looks like this:

```
CREATE TABLE my_outbox
(
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  aggregate_type    VARCHAR(255) NOT NULL,
  aggregate_id      VARCHAR(255) NOT NULL,
  event_type        VARCHAR(255) NOT NULL,
  payload           JSONB NOT NULL,
  created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  processed         BOOLEAN DEFAULT FALSE
);
```

Example 7-25: The fields serve the following purposes:

<code>id:</code>	A unique identifier for each event, used for idempotency checking in consumers and for deduplication.
<code>aggregate_type:</code>	The type of the business entity that changed (e.g., "Order", "Customer", "Product"). In DDD terminology, the aggregate root type.
<code>aggregate_id:</code>	The identifier of the specific entity instance (e.g., the order ID, customer ID, or product SKU).
<code>event_type:</code>	The name of the business event (e.g., "OrderCreated", "OrderShipped", "CustomerUpdated").
<code>payload:</code>	The event data as a JSONB object. This is what downstream consumers receive.
<code>created_at:</code>	The timestamp when the event was written to the outbox.
<code>processed:</code>	An optional flag used in polling-based outbox implementations. For CDC-based delivery, this column is not strictly necessary because CDC tracks position in the WAL, not in the table.

In the demo applications in this document, the `processed` column is omitted and the `payload` is populated with the application data directly.

7.1.7 Transaction Ordering and Foreign Key Considerations

CDC delivers row-level events in the order they are committed to the WAL. Within a single table, ordering is straightforward: events appear in commit order. Across multiple tables, and especially in a distributed database where different tables may reside on different tablets or nodes, ordering becomes more nuanced. This section addresses the practical implications of CDC event ordering and the foreign key problem that arises in consumer databases.

Row-Level Events, Not Statement-Level A CDC consumer receives events at the row level. A single SQL statement that modifies 500 rows generates 500 CDC events. A multi-table transaction generates events for every modified row in every modified table. The consumer sees these as a sequence of individual row-change events.

This means that consumers must handle the following realities:

- A parent-child relationship (orders and order_items) may generate CDC events in a sequence where child events arrive before the parent event.
- A transaction that modifies 1,000 rows may produce 1,000 events that all share the same transaction ID but arrive over time as the CDC consumer processes them.
- An UPDATE that touches the same row multiple times within a transaction may generate multiple events for that row (depending on CDC configuration and snapshot settings).

The Foreign Key Problem The most common ordering problem in CDC consumer databases is the foreign key violation. Suppose you have an orders table and an order_items table, with a foreign key from order_items.order_id to orders.id.

Example 7-26: In the source database, the application inserts a new order in a transaction:

```
BEGIN;
INSERT INTO orders (id, customer_id, total) VALUES (42, 101, 99.99);
INSERT INTO order_items (order_id, product_id, qty) VALUES (42, 5, 2);
INSERT INTO order_items (order_id, product_id, qty) VALUES (42, 8, 1);
COMMIT;
```

These three rows are committed atomically. In the CDC stream, they produce three events, all with the same transaction ID. But the CDC consumer may apply them to its own database in an order that puts the order_items events before the orders event. If the consumer database has the same foreign key constraint, the order_items INSERT fails because the referenced orders row does not yet exist.

Solutions to the Foreign Key Problem There are three common approaches:

Disable FK constraints on the consumer:

This is the most common approach in CDC target databases. The source database enforces referential integrity. The consumer database is a replica or downstream store that trusts the source to be consistent. Disabling FK constraints on the consumer eliminates the ordering problem entirely.

Deferrable constraints:

PostgreSQL (and yugabyteDB) support deferrable foreign key constraints. With DEFERRABLE INITIALLY DEFERRED, the constraint is checked at commit time rather than at each row insertion. A CDC consumer can apply all events from a transaction inside a single SQL transaction with deferred constraints, and the integrity check happens at commit, by which time all the rows are present.

Example 7-27: Deferrable constraints

```
ALTER TABLE order_items
ADD CONSTRAINT fk_order
FOREIGN KEY (order_id) REFERENCES orders(id)
DEFERRABLE INITIALLY DEFERRED;
```

Example 7-28: In the CDC consumer code:

```
BEGIN;
SET CONSTRAINTS ALL DEFERRED;
-- apply all events from this transaction in any order
COMMIT; -- constraint check happens here
```

Buffer and sort by transaction:

The consumer reads events and groups them by transaction ID. Once a complete transaction is buffered (all events sharing the same xid have arrived), the consumer applies them in dependency order (parents before children). This approach requires the consumer to detect when a transaction is complete, which is possible with wal2json format version 2 (each message includes total event count for its transaction) or with Debezium's transaction metadata.

Transaction Metadata in wal2json and Debezium Both CDC paths provide transaction metadata that consumers can use for grouping and ordering.

In wal2json with include-xids enabled, every change event includes the xid (transaction ID):

Example 7-29: wal2json xid's

```
{ "xid": 1234, "change": [ { "kind": "insert", "table": "orders", ... } ] }
{ "xid": 1234, "change": [ { "kind": "insert", "table": "order_items", ... } ] }
```

Both events share xid 1234, allowing a consumer to group them together.

In Debezium, the transaction metadata (when `provide.transaction.metadata=true` is set) includes:

```
transaction.id:                The unique transaction identifier
transaction.total_order:       The position of this event within the entire transaction
transaction.data_collection_order: The position within this table within the transaction
```

This metadata allows consumers to reconstruct the exact ordering of events within a transaction and to know when a transaction is complete (`total_order` equals `total_events_in_transaction`).

[!NOTE] The gRPC CDC path reads from tablets in parallel. Because different rows of the same transaction may reside on different tablets (in a distributed database, a single transaction often touches multiple tablets), events from the same transaction may arrive from different tablet streams at unpredictable times. The PostgreSQL Logical Replication path, despite its lower throughput, produces a single ordered stream coordinated at the database level. For consumers that require strict transaction ordering, PG Logical Replication actually provides a simpler ordering guarantee than gRPC CDC.

Deferrable Constraints: Complete Example Example 7-30: Here is a complete example of using deferrable constraints in a CDC consumer that needs to handle parent-child ordering:

```
-- On the consumer database, define the FK as deferrable
ALTER TABLE order_items
  ADD CONSTRAINT fk_order
  FOREIGN KEY (order_id) REFERENCES orders(id)
  DEFERRABLE INITIALLY DEFERRED;

-- In the CDC consumer, wrap each transaction's events:
BEGIN;
SET CONSTRAINTS ALL DEFERRED;

-- Apply events in any order
INSERT INTO orders (id, customer_id, total)
  VALUES (42, 101, 99.99)
  ON CONFLICT (id) DO NOTHING;

INSERT INTO order_items (order_id, product_id, qty)
  VALUES (42, 5, 2)
  ON CONFLICT DO NOTHING;

INSERT INTO order_items (order_id, product_id, qty)
  VALUES (42, 8, 1)
  ON CONFLICT DO NOTHING;

COMMIT; -- FK check happens here; all rows present, check passes
```

The SET CONSTRAINTS ALL DEFERRED statement moves all deferrable constraint checks to the end of the transaction. As long as all events from the same source transaction are applied within the same consumer transaction, the parent row will be present by the time the commit-time check runs.

This approach requires buffering events by transaction ID and applying them together, but it preserves referential integrity in the consumer database without disabling FK constraints entirely.

7.2 Complete the following

In this section, we set up and run two working CDC demonstration applications that show both paths in action. Each demo uses a Python Flask application as the data source, writing to a yugabyteDB database, and demonstrates a different approach to consuming the resulting change stream.

Demo A demonstrates gRPC CDC with Debezium Server. A Python Flask source application writes to a yugabyteDB database. Those writes include an insert into an outbox table in the same transaction as the business data. Debezium Server reads the CDC stream, captures changes to the outbox table, and delivers those changes via HTTP POST to a Python Flask receiver application. No Kafka is involved. The Debezium Server is a standalone Java process.

Demo B demonstrates PostgreSQL Logical Replication without Debezium, Kafka, or Java. A Python Flask source application writes to yugabyteDB. A Python consumer connects directly using psycopg2's LogicalReplicationConnection and displays every INSERT, UPDATE, and DELETE in a live web UI. The entire stack is Python.

Together, the two demos show both the high-throughput, Debezium-based approach and the lightweight, Python-native approach.

7.2.1 Prerequisites

The following prerequisites apply to both demos unless otherwise noted.

A Running yugabyteDB Cluster Both demos require a running yugabyteDB cluster. A three-node cluster with replication factor 3 is recommended for production-like behavior, but a single-node cluster works for demonstration purposes.

Example 7-31: For a single-node development cluster, you can start yugabyteDB with:

```
./bin/yugabyted start --advertise_address 127.0.0.1
```

For a multi-node cluster, refer to the yugabyteDB documentation for your deployment environment.

Verify wal{__}level **Example 7-32:** Connect to the yugabyteDB cluster using `ysqlsh` or `psql` and verify that `wal_level` is set to `logical`:

```
SHOW wal_level;
```

Example 7-33: The expected output is:

```
wal_level
-----
logical
```

If the output is 'replica' or 'minimal', the cluster must be reconfigured with logical WAL enabled. In yugabyteDB, this requires setting the GFlag `-ysql_enable_replication_slot=true` (or the equivalent per your version) and restarting the cluster.

Verify Replication Privilege **Example 7-34:** The database user used for CDC must have the `REPLICATION` privilege. Verify the yugabyte superuser's replication status:

```
SELECT rolname, rolreplication FROM pg_roles WHERE rolname = 'yugabyte';
```

Example 7-35: Expected output:

```
rolname | rolreplication
-----+-----
yugabyte | t
```

Example 7-36: If rolreplication is ‘f’ (false), grant the privilege:

```
ALTER ROLE yugabyte WITH REPLICATION;
```

Python Environment **Example 7-37: Both demos require Python 3.x with the following packages installed:**

Both demos require Python 3.x with the following packages installed:

Example 7-38: Verify the installation:

```
python3 -c "import psycpg2, flask; print('OK')"
```

Network Access The machine running Debezium Server (Demo A only) must have network access to the yugabyteDB cluster on the following ports:

```
-- Master ports: 7100 (used by yb-admin and the gRPC CDC connection for stream management)
-- TServer ports: 5433 (YSQL/PostgreSQL protocol) and 9100 (gRPC internal tablet protocol, used by the g
```

If running in a cloud environment or behind a firewall, ensure these ports are open between the Debezium Server host and all cluster nodes.

Demo A Specific: Java 21 Java 21 is required for Debezium Server. Java 25 is not compatible with Debezium 1.9.5. Another section of this document covers the installation of Java 21 from Eclipse Temurin.

Example 7-39: Checking the Java version

```
java -version -- should show openjdk version "21.x.x"
```

Demo A Specific: yb-admin CLI The yb-admin CLI ships with every yugabyteDB installation and is located in the bin directory. You will use it to create, list, inspect, and delete CDC streams. Ensure it is on your PATH or note the full path to the binary.

Example 7-40: Locations of yb-admin

```
which yb-admin -- if installed to PATH

./bin/yb-admin --help -- if using the local yugabyteDB installation
```

7.2.2 Demo A: gRPC CDC with Debezium Server

7.2.2.1 What We Will Accomplish Demo A shows the gRPC CDC path from end to end. When complete, you will have a working pipeline in which:

1. A Python Flask source application accepts HTTP requests that trigger inserts, updates, and deletes against the yugabyteDB t1 table.
2. Each write to t1 is accompanied by an insert into the my_outbox table in the same database transaction. The outbox insert carries a JSON payload with the event data.
3. A CDC stream (created with yb-admin) watches the database. Debezium Server is configured to consume that stream and filter to the

my_outbox table only.

4. Debezium Server reads every new row inserted into my_outbox and delivers it via HTTP POST to a Python Flask receiver application.
5. The receiver application logs the event, formats it for display, and makes it available via a web interface.

The key design feature demonstrated here is the Outbox Pattern: Debezium sees only the outbox table, not the business table (t1). The event payload in the outbox is explicitly designed for downstream consumers.

The architecture for this demo is:

Source App (Flask) --> yugabyteDB (t1 + my_outbox) --> gRPC CDC Stream --> Debezium Server --> H

All components run on a single machine for this demonstration. In production, the source application and receiver application would be separate services.

7.2.2.2 Step 1: Create the Database Tables Example 7-41: Connect to yugabyteDB and create the required tables. The t1 table is the business table. The my_outbox table is the CDC-watched outbox.

```
-- Drop and recreate if running a fresh start
DROP TABLE IF EXISTS my_outbox;
DROP TABLE IF EXISTS t1;

-- Business table
CREATE TABLE t1
(
  col1 VARCHAR(50) PRIMARY KEY,
  col2 VARCHAR(50),
  col3 VARCHAR(50),
  col4 VARCHAR(50)
);

-- Outbox table for CDC delivery
CREATE TABLE my_outbox
(
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  aggregate_type    VARCHAR(255) NOT NULL,
  aggregate_id      VARCHAR(255) NOT NULL,
  event_type        VARCHAR(255) NOT NULL,
  payload            JSONB NOT NULL,
  created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Note that the my_outbox table does not have a processed column. In this implementation, CDC tracks position in the WAL via the replication stream, not via a column on the outbox table.

The t1 table uses col1 as the primary key. The source application performs upserts using ON CONFLICT (col1) DO UPDATE, so repeated calls with the same col1 value update the row rather than failing on a duplicate key.

7.2.2.3 Step 2: Create the CDC Stream Use yb-admin to create a CDC stream for the database. The stream is created at the database level and will capture changes to all tables (Debezium is configured to filter to my_outbox specifically).

Example 7-42: yb-admin command

```
yb-admin \
--master_addresses master1:7100,master2:7100,master3:7100 \
create_change_data_stream ysql.yugabyte
```

Replace master1, master2, master3 with your actual master node addresses, and replace yugabyte with your database name if different.

Example 7-43: The command output will include a line like:

```
CDC Stream ID: d540f5e4802b4d9589aeedea62d03079
```

Save this Stream ID. You will use it in the Debezium application.properties file. The exact UUID you receive will differ from this example.

Managing CDC Streams Example 7-44: yb-admin provides several commands for managing CDC streams:

List all streams:

```
yb-admin \
--master_addresses master1:7100,master2:7100,master3:7100 \
list_change_data_streams
```

Get details about a specific stream:

```
yb-admin \
--master_addresses master1:7100,master2:7100,master3:7100 \
get_change_data_stream_info <stream_id>
```

Delete a stream:

```
yb-admin \
--master_addresses master1:7100,master2:7100,master3:7100 \
delete_change_data_stream <stream_id>
```

[!NOTE] CDC streams in the gRPC path are persistent cluster-level objects. Unlike replication slots, they do not automatically accumulate unbounded WAL because the gRPC CDC path uses a different mechanism for WAL retention. However, you should still clean up streams that are no longer in use. The list_change_data_streams command is useful for auditing which streams exist.

7.2.2.4 Step 3: Install Java 21 Debezium Server 1.9.5 requires Java 21. Use Eclipse Temurin 21 LTS, which is freely available and widely used.

Example 7-45: First, check whether Java 21 is already installed:

```
java -version
```

a different version, install Temurin 21 as follows.

On Ubuntu/Debian:

```
# Add the Adoptium repository
```

```
wget -O - https://packages.adoptium.net/artifactory/api/gpg/key/public \
| sudo apt-key add -
```

```
echo "deb https://packages.adoptium.net/artifactory/deb \
$(awk -F= '/^VERSION_CODENAME/{print$2}' /etc/os-release) main" \
| sudo tee /etc/apt/sources.list.d/adoptium.list
```

```
sudo apt-get update
```

```
sudo apt-get install -y temurin-21-jdk
```

On RHEL/Rocky/AlmaLinux:

```
sudo rpm --import https://packages.adoptium.net/artifactory/api/gpg/key/public

# Create the repository file
sudo tee /etc/yum.repos.d/adoptium.repo << 'EOF'
[Adoptium]
name=Adoptium
baseurl=https://packages.adoptium.net/artifactory/rpm/rhel/$releasever/$basearch
enabled=1
gpgcheck=1
gpgkey=https://packages.adoptium.net/artifactory/api/gpg/key/public
EOF

sudo dnf install -y temurin-21-jdk
```

Verify the installation: Example 7-46: Getting the Java version.

```
java -version
# Expected: openjdk version "21.x.x" ...
```

Example 7-47: If multiple Java versions are installed, set `JAVA_HOME` explicitly when running Debezium:

```
export JAVA_HOME=/usr/lib/jvm/temurin-21-amd64 # adjust path as needed
export PATH=$JAVA_HOME/bin:$PATH
```

7.2.2.5 Step 4: Install Debezium Server 1.9.5 Download Debezium Server 1.9.5 from Maven Central. This is the standalone server distribution that requires no Kafka.

```
# Create an installation directory
mkdir -p ~/debezium-server
cd ~/debezium-server

# Download Debezium Server 1.9.5
wget https://repo1.maven.org/maven2/io/debezium/debezium-server-dist/\
1.9.5.Final/debezium-server-dist-1.9.5.Final.tar.gz

# Extract the archive
tar xf debezium-server-dist-1.9.5.Final.tar.gz

# Create the data directory (used for offset and schema history files)
mkdir -p debezium-server/data
```

The extraction creates a `debezium-server` directory containing:

```
debezium-server/run.sh      -- the startup script
debezium-server/conf/      -- configuration files go here
debezium-server/lib/       -- the server JARs
```

The data directory will hold the offset tracking file (`offsets.dat`) and the schema history file (`schema_history.dat`). These are how Debezium Server tracks its position in the CDC stream between restarts.

7.2.2.6 Step 5: Install the yugabyteDB gRPC Connector JAR The yugabyteDB gRPC CDC connector is not bundled with Debezium Server. It must be downloaded separately from the yugabyte/debezium-connector-yugabytedb GitHub repository.

```
cd ~/debezium-server/debezium-server
```

```
# Download the connector JAR
# Visit https://github.com/yugabyte/debezium-connector-yugabytedb/releases
# to find the latest release for your yugabyteDB version.
# Example URL (replace with the current release):
wget https://github.com/yugabyte/debezium-connector-yugabytedb/\
releases/download/v1.9.5.y.30/
debezium-connector-yugabytedb-1.9.5.y.30.jar
-O lib/debezium-connector-yugabytedb.jar
```

Note:

The connector JAR version must be compatible with both your yugabyteDB version and Debezium Server 1.9.5.

Place the JAR file in the lib directory alongside the other Debezium JARs. Debezium Server automatically loads all JARs from the lib directory at startup.

7.2.2.7 Step 6: Configure Debezium Create the Debezium Server configuration file at `~/debezium-server/debezium-server/conf/application.properties`. This file tells Debezium Server where to connect, what to consume, and where to deliver events.

```
# ~/debezium-server/debezium-server/conf/application.properties
```

```
# Sink configuration: deliver events via HTTP POST
debezium.sink.type=http
debezium.sink.http.url=http://127.0.0.1:5020/events
```

```
# Source connector: yugabyteDB gRPC CDC
debezium.source.connector.class=\
io.debezium.connector.yugabytedb.YugabyteDBGRPCConnector
```

```
# Database connection
debezium.source.database.hostname=<host>
debezium.source.database.port=5433
debezium.source.database.user=yugabyte
debezium.source.database.password=<password>
debezium.source.database.dbname=<dbname>
```

```
# Cluster master addresses (for CDC stream management)
debezium.source.database.master.addresses=\
:7100,:7100,:7100
```

```
# The Stream ID from Step 2
debezium.source.database.streamid=<stream_id>
```

```
# Watch only the outbox table
debezium.source.table.include.list=public.my_outbox
```

```
# Logical server name (used as a prefix in topic/event naming)
debezium.source.database.server.name=outbox_cdc
```

```
# Offset storage (tracks position in the CDC stream)
debezium.source.offset.storage=\
```

```
org.apache.kafka.connect.storage.FileOffsetBackingStore debezium.source.offset.storage.file.filename=data/offsets.dat
debezium.source.offset.flush.interval.ms=0

# Schema history (tracks table schema over time)
debezium.source.schema.history.internal=\

io.debezium.storage.file.history.FileSchemaHistory debezium.source.schema.history.internal.file.filename=data/schema_history

# Include transaction metadata in events
debezium.source.provide.transaction.metadata=true

# Use JSON format for both keys and values
debezium.format.value=json
debezium.format.key=json
```

Replace the placeholder values:

```
<host>          Your yugabyteDB cluster hostname or IP address
<password>      The password for the yugabyte user (or empty if none)
<dbname>        Your database name
<master1-3>     Your master node addresses
<stream_id>     The Stream ID from Step 2
```

Configuration Notes The `debezium.source.table.include.list` parameter is crucial. By specifying `public.my_outbox`, you tell Debezium to emit events only for the outbox table. Changes to `t1` and any other tables are not delivered, even though the CDC stream covers the entire database. This is the correct pattern: CDC watches the whole database, but Debezium filters to only the outbox table.

The `debezium.source.offset.flush.interval.ms=0` setting causes Debezium to flush its offset (the WAL position it has consumed) immediately after each event. This is the safest setting for consistency: if Debezium crashes, it restarts from the last flushed offset with no events skipped. A non-zero value improves throughput at the cost of potentially redelivering some events on restart.

The `data/offsets.dat` and `data/schema_history.dat` paths are relative to the directory from which Debezium Server is run (the `debezium-server` directory). Ensure the data directory exists before starting Debezium.

7.2.2.8 Step 7: The Source Application (60_index.py) The source application is a Python Flask web application that provides an interface for writing rows to the yugabyteDB `t1` table. Each write also inserts a row into the `my_outbox` table in the same database transaction. This is the Outbox Pattern in action.

The application is called `60_index.py` in the demo project directory. It exposes a web form at `http://127.0.0.1:5000/` where you can submit new rows or trigger deletes.

The Atomic Write Pattern Example 7-48: The critical pattern is the double-write within a single transaction. Here is the relevant code from `60_index.py`:

```
l_db.autocommit = False
l_cur = l_db.cursor()

# Insert the business data into t1
l_cur.execute(
    "INSERT INTO t1 (col1, col2, col3, col4) VALUES (%s, %s, %s, %s) "
    "ON CONFLICT (col1) DO UPDATE SET col2 = EXCLUDED.col2, "
    "col3 = EXCLUDED.col3, col4 = EXCLUDED.col4",
    (col1, col2, col3, col4),
)
```

```
# Insert event to outbox (in SAME transaction)
l_payload = json.dumps({"col1": col1, "col2": col2,
    "col3": col3, "col4": col4})

l_cur.execute(
    "INSERT INTO my_outbox (aggregate_type, aggregate_id, "
    "event_type, payload, created_at) "
    "VALUES (%s, %s, %s, %s::jsonb, CURRENT_TIMESTAMP)",
    ("T1", col1, "T1RowCreated", l_payload),
)

l_db.commit() # Both writes commit atomically
```

The key points of this pattern:

- autocommit is disabled, so both INSERTs are in the same transaction.
- The t1 INSERT uses ON CONFLICT ... DO UPDATE (an upsert), so running the form twice with the same col1 value updates t1 rather than failing.
- The my_outbox INSERT always creates a new row with a new UUID, so every write to t1 produces a new outbox event, regardless of whether t1 was inserted or updated.
- The single l_db.commit() commits both changes atomically. If anything fails between the two INSERTs, both are rolled back. The outbox never contains an event for a write that did not also commit to t1.

The payload is a JSON string constructed from the column values and cast to JSONB in the database. This means the payload is validated as valid JSON at insert time, and it is stored efficiently as a binary JSON value.

What Happens After the Commit

After `l_db.commit()` executes:

1. yugabyteDB writes both the t1 row and the my_outbox row to its WAL.
2. The gRPC CDC stream sees the new my_outbox row.
3. Debezium Server receives the CDC event for my_outbox.
4. Debezium formats the event using the change event envelope.
5. Debezium POSTs the event to `http://127.0.0.1:5020/events`.
6. The receiver application (`80_debezium_client.py`) processes the event.

7.2.2.9 Step 8: The Event Receiver Application (`80_debezium_client.py`) The receiver application is a Python Flask web application that listens for HTTP POST requests from Debezium Server on port 5020. It receives the raw Debezium event envelope, extracts the relevant fields, and displays them in a web interface.

The application is called `80_debezium_client.py` in the demo project directory.

The `/events` Endpoint Debezium Server posts each change event to the URL configured in `debezium.sink.http.url` (`http://127.0.0.1:5020/events`). The receiver application's `/events` route accepts these POSTs:

```
@app.route('/events', methods=['POST'])
def receive_event():
    raw = request.get_json(force=True, silent=True) or {}
    event_str = format_event(raw)
```

```
# Store and display the formatted event
...
return '', 200
```

The route returns HTTP 200 immediately. Debezium Server interprets any 2xx response as successful delivery. If the receiver returns a non-2xx status, Debezium retries delivery according to its retry configuration.

The `format{__}event()` Function The `format_event()` function extracts meaningful fields from the Debezium event envelope. The envelope nests the actual row data inside several levels of JSON:

```
-- The top level contains the Debezium schema and payload fields.
-- The payload contains before, after, source, op, and ts_ms.
-- The after field contains the row data as a nested object with
   key-value pairs for each column.
```

For my_outbox CDC events, the `after.payload` field (a column in the outbox table named `payload`, not the Debezium payload) contains the business event data.

The `format_event()` function navigates this structure and extracts:

```
-- The operation type (INSERT, UPDATE, DELETE)
-- The after-state of the outbox row
-- The payload JSON (the business event data)
-- The aggregate_type and event_type
-- The created_at timestamp
```

These fields are formatted into a human-readable string for display in the web interface.

Section summary In this section, we built and ran both CDC demonstrations end to end. Demo A wired a Python Flask source application to yugabyteDB's outbox table, ran Debezium Server against the gRPC CDC stream, and delivered every business event to a Flask receiver over plain HTTP – no Kafka, no message broker, just Debezium Server and two small Python applications. Demo B replaced Debezium entirely: a Python consumer connected directly to yugabyteDB using `psycpg2`'s `LogicalReplicationConnection` and rendered every INSERT, UPDATE, and DELETE in a live web UI, with no Java process in the stack at all. Between the two, we exercised both of yugabyteDB's CDC subsystems against a real, running cluster, not just in theory.

7.3 In this document, we reviewed or created

This month and in this document we detailed the following:

- The two change-data-capture subsystems built into yugabyteDB – gRPC CDC (consumed via Debezium Server, with per-tablet parallelism suited to high-throughput workloads) and PostgreSQL Logical Replication (consumed with standard PostgreSQL tooling, including a direct `psycpg2` connection with no additional connector required) – along with the tradeoffs that should drive a choice between them.
- Two complete, runnable demonstration applications: a Debezium-based outbox-pattern pipeline delivering business events over HTTP with no Kafka involved, and a Kafka-free, Debezium-free, Java-free Python application that consumes yugabyteDB's logical replication stream directly and displays every row-level change in a live web UI.

Persons who helped this month Kiyu Gabriel, David Bechberger

Additional resources: Free yugabyteDB training courses,

https://university.yugabyte.com/users/sign_in

Take any class, anytime, for free.

Jim Knicely's very excellent blog site,
<https://yugabytedb.tips/>

This document is located here,

<https://www.yugabitten.com/62%20-%20Monthly%20articles/2026-07%20-%20Change%20Data%20Capture/07%20-%20Change%20data%20capture.pdf>